São Paulo Tech School

Gustavo Lima de Araujo

Case ITOPS

Evidências da criação, configuração e resultados da atividade “Case ITOPS”, no qual, foi criado uma simples ETL e a Arquitetura Medalhão no S3.

Rua Haddock Lobo, 595

10.2026

Gustavo Lima de Araujo

Case ITOPS

Evidências da criação, configuração e resultados da atividade “Case ITOPS”, no qual, foi criado uma simples ETL e a Arquitetura Medalhão no S3.

Este documento tem como objetivo estudar e enfatizar conceitos sobre a criação de buckets e os seus prefixos no recurso S3 da AWS, além da criação de algumas ETLs que se conectam nos prefixos dos buckets e fazem uma certa analise e manipulação dos dados.

Orientador: Eduardo Verri

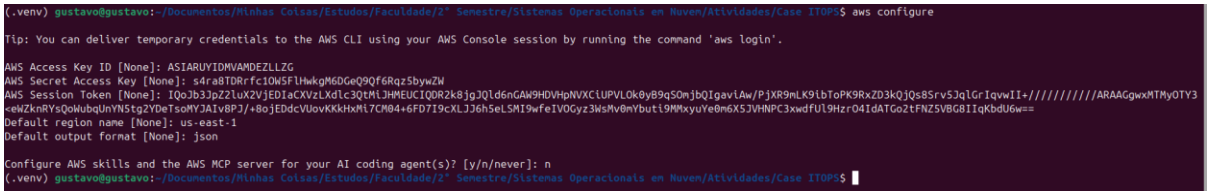
Rua Haddock Lobo, 595

10.2026

Sumário

1. Configurando a ferramenta AWS CLI 3
2. Criação de Buckets 4
3. Criando prefixos no Bucket 7
4. Criando uma Instância na EC2 9
5. Configurando instância para receber arquivos python 12
6. Arquivos Python 13
   6.1. captura-access-point.py 14
   6.2. captura-firewall.py 15
   6.3. bronze2silver.py 18
   6.4. silver2gold.py 21
7. Enviando os arquivos python e .env para a instância 25
8. Configurando python e venv na instância 27
9. Inserindo dados no Bucket 28
10. Criando CSV a partir dos arquivos python na instância 30
11. Configurando o contab 32
12. CSV Final 33

1. Configurando a ferramenta AWS CLI

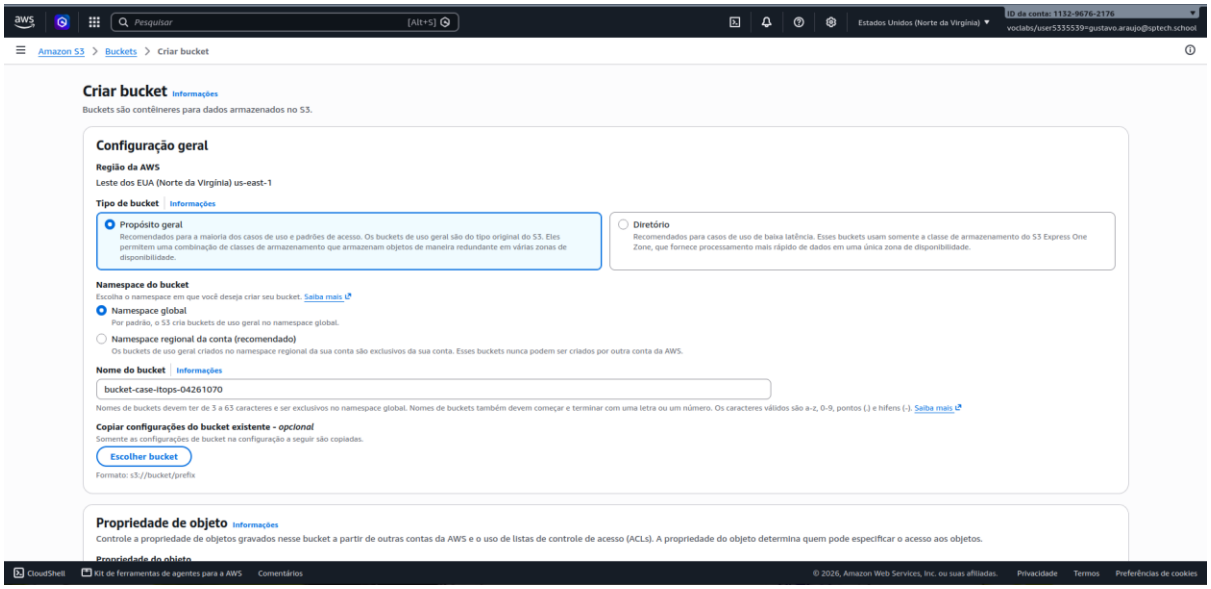


Inicio o procedimento configurando as credenciais de acesso do cliente de linha de comando da AWS. Executo o comando aws configure via terminal e informo as credenciais temporárias de autenticação, incluindo o AWS Access Key ID, o AWS Secret Access Key e o AWS Session Token. Além disso, defino a região padrão de operação como us-east-1 (Norte da Virgínia) e o formato de saída das respostas em json.

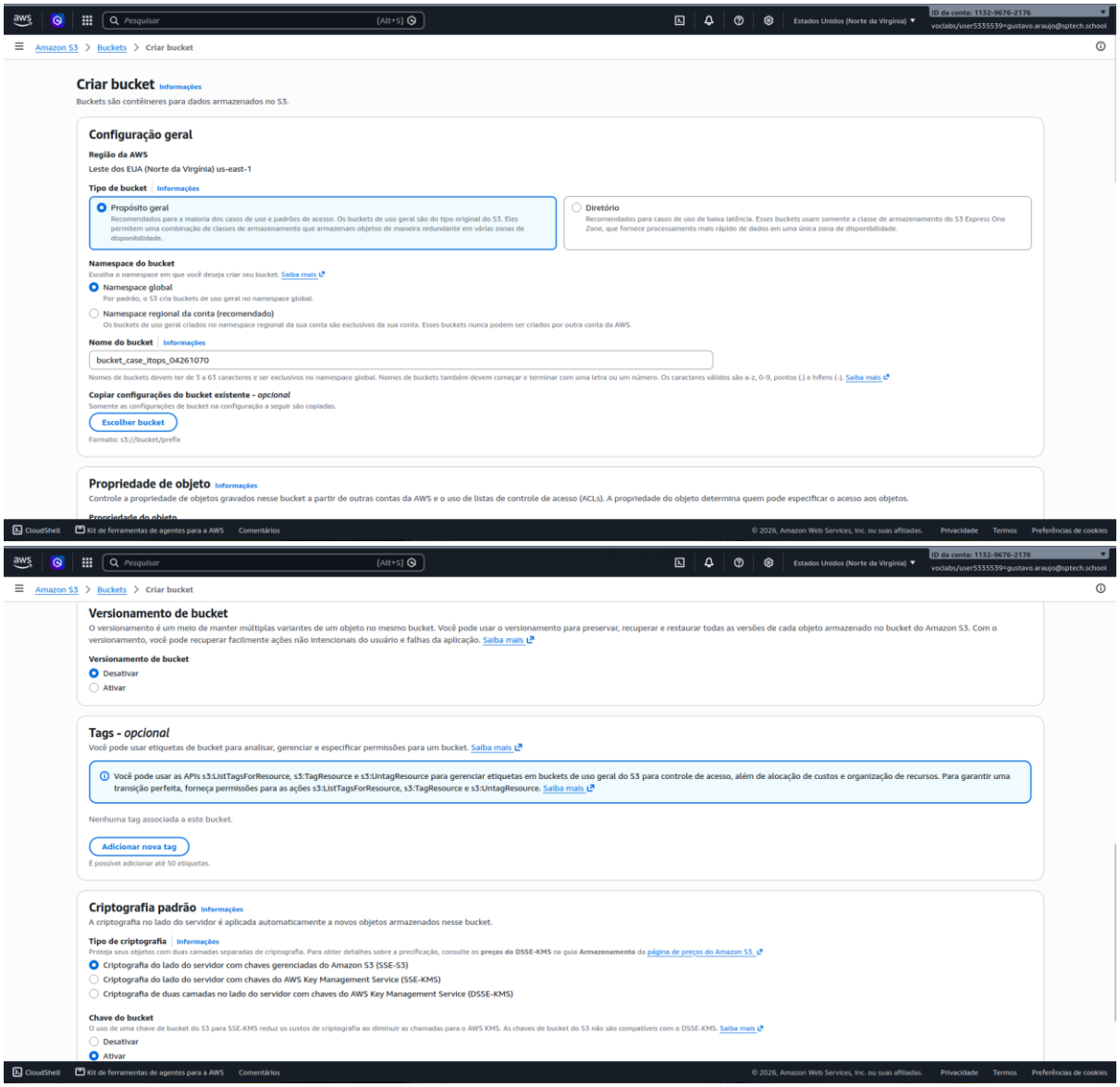


Para validar a autenticação e confirmar se o perfil foi configurado corretamente, executo o comando aws sts get-caller-identity. O retorno da chamada confirma a identidade do usuário ativo, o ID da conta AWS e a *Role* atribuída no ambiente.

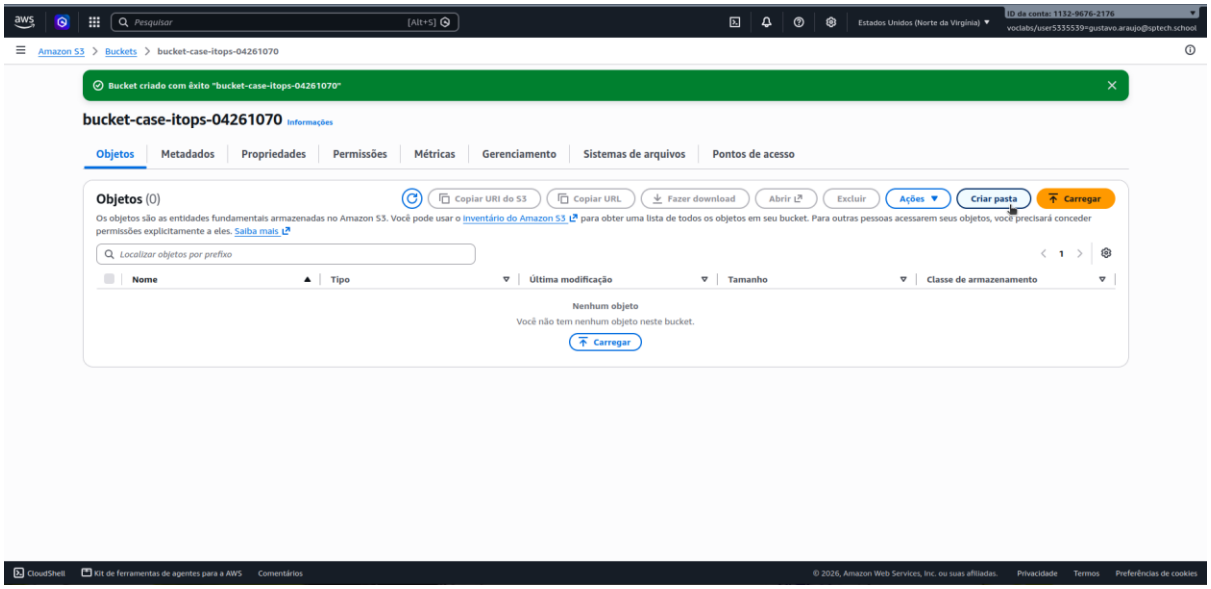
2. Criação de Buckets



Acesso o console de gerenciamento do Amazon S3 para iniciar a criação de um novo recurso de armazenamento. Na etapa de Configuração geral, mantenho a região selecionada como us-east-1, escolho o tipo de bucket como Propósito geral com namespace global e defino o nome do bucket como bucket-case-itops-04261070.

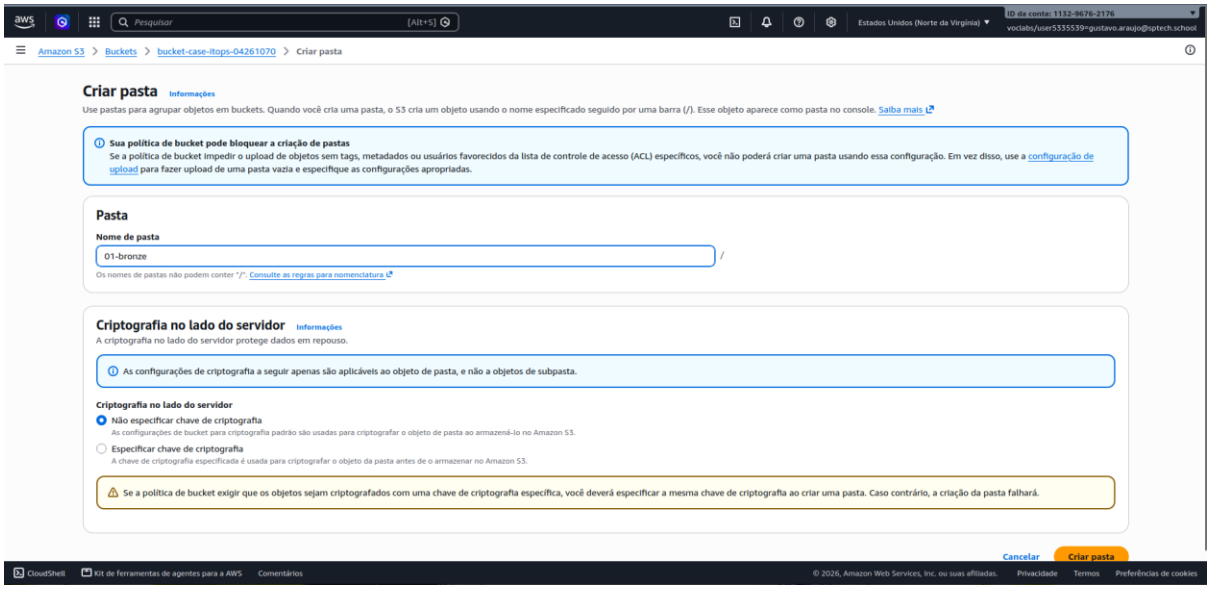


Reviso os parâmetros de configuração geral do bucket bucket-case-itops-04261070 antes de prosseguir para as políticas de acesso e segurança, garantindo que o nome do bucket atenda aos padrões globais de nomenclatura e esteja alocado na região correta (us-east-1).

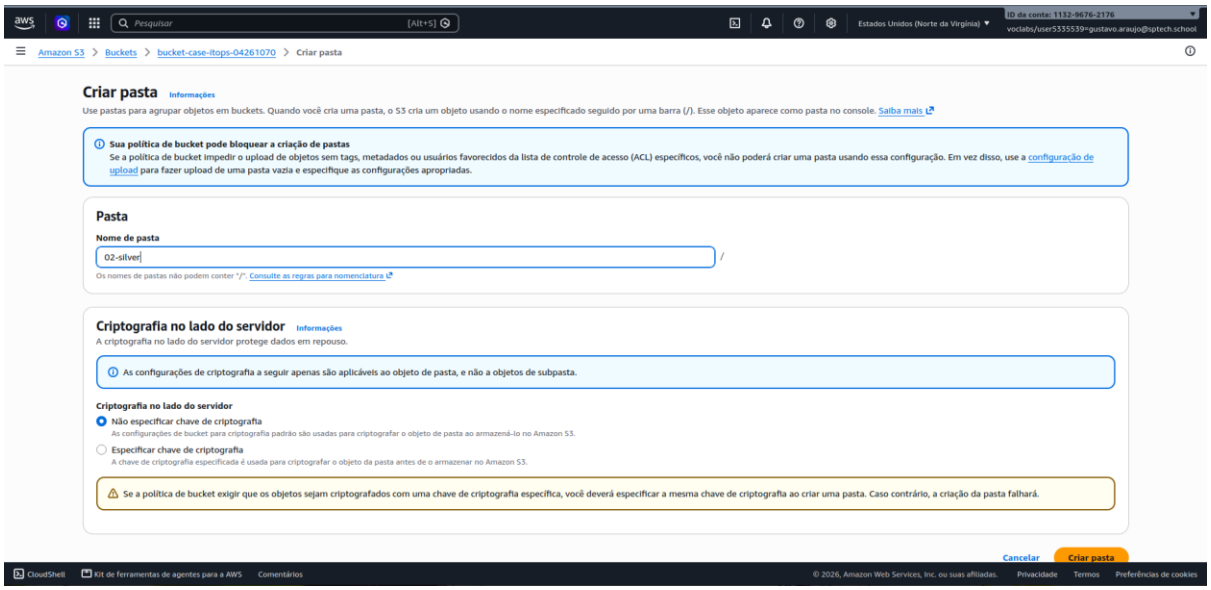


Após a confirmação de que o bucket bucket-case-itops-04261070 foi criado com sucesso no Amazon S3, acesso a aba Objetos do bucket recém-criado e clico na opção Criar pasta para dar início ao processo de organização da arquitetura de diretórios do ambiente.

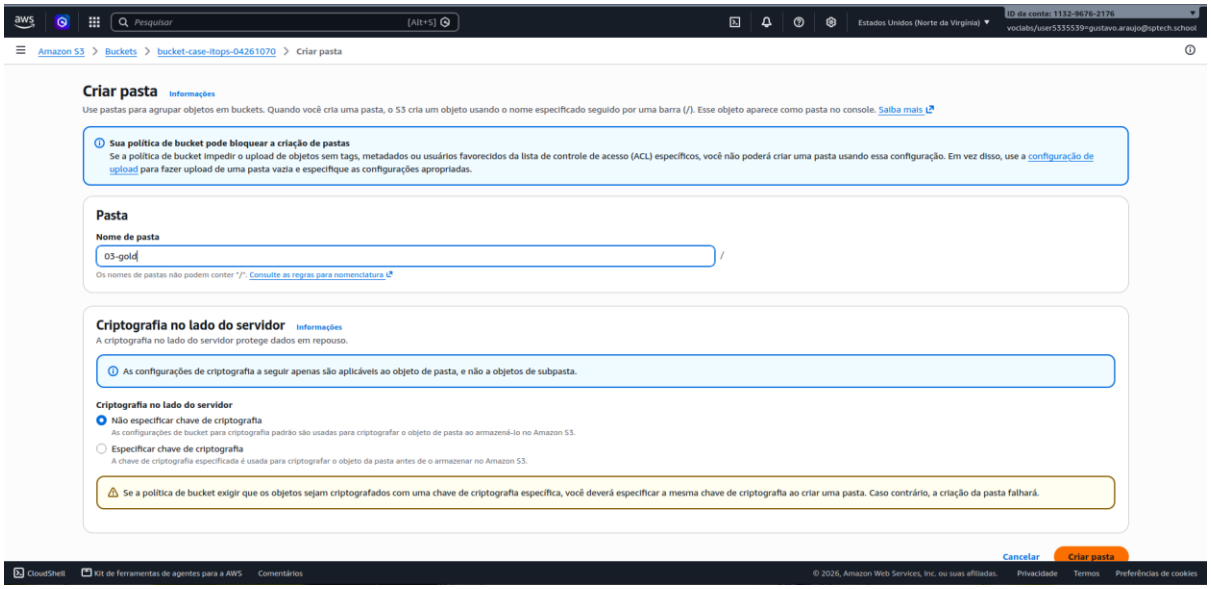
3. Criando prefixos no Bucket



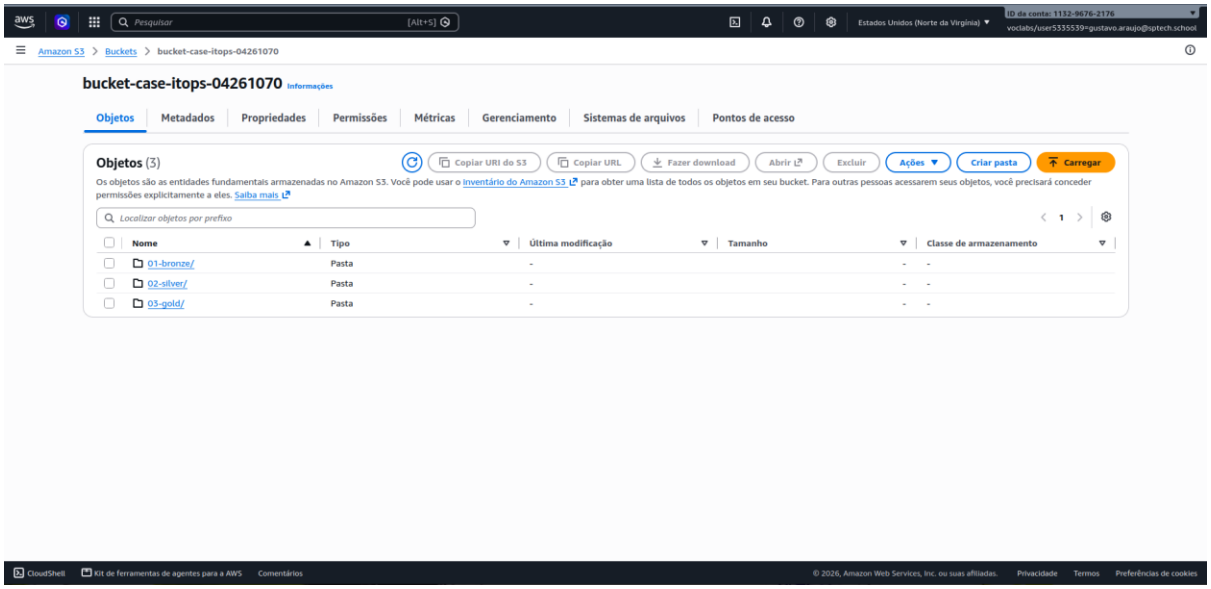
Início a estruturação das camadas de dados arquitetura de Medalhão no bucket. Na tela de criação de pasta, defino a primeira camada informando o nome 01-bronze, mantendo as configurações padrão de criptografia do lado do servidor e confirmando a ação no botão Criar pasta.



Prosseguindo com o isolamento das camadas de processamento de dados, realizo a criação da segunda pasta dentro do mesmo bucket, atribuindo o nome 02-silver e mantendo o padrão de criptografia herdado do bucket.

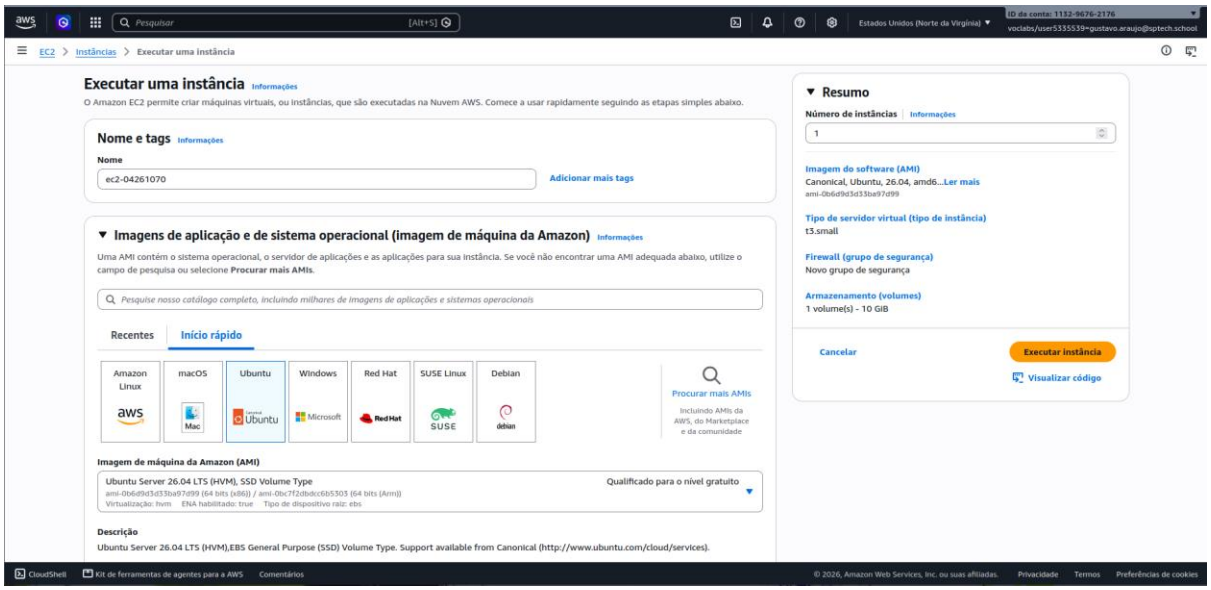


Por fim, concluo a organização do bucket criando a terceira pasta necessária para a arquitetura, definindo o nome como 03-gold. Com isso, finalizo a estrutura completa de diretórios em camadas (Bronze, Silver e Gold) no Amazon S3.

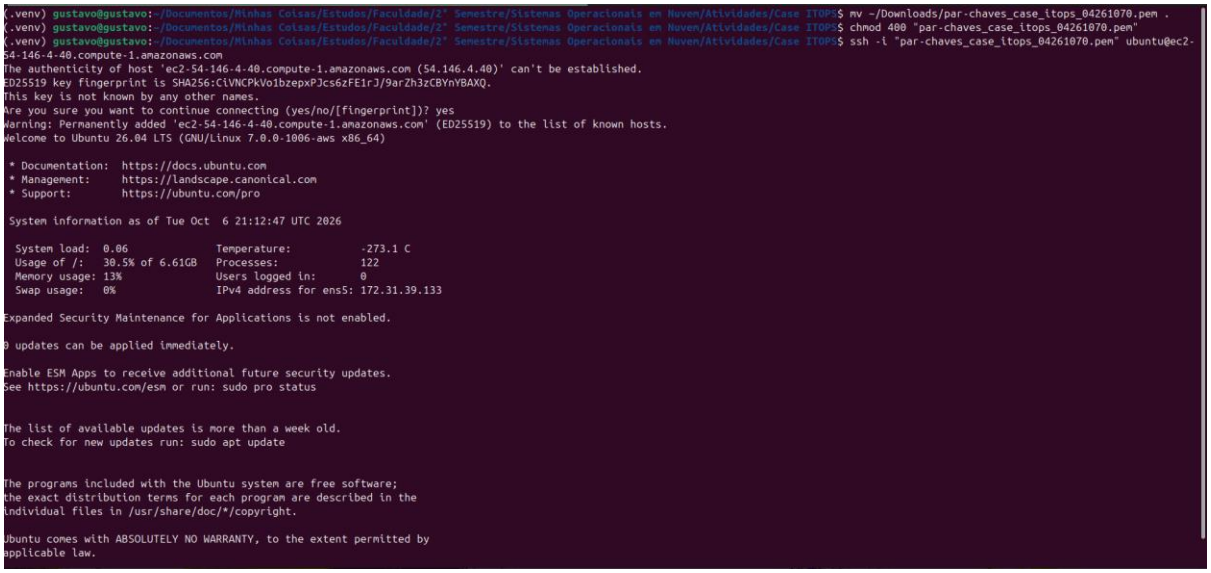


Verifico a estrutura final do bucket bucket-case-itops-04261070 na aba de objetos do console do Amazon S3. Confirmo que as três pastas da arquitetura (01-bronze/, 02-silver/ e 03-gold/) foram criadas com sucesso, estabelecendo a base para o armazenamento de dados do projeto.

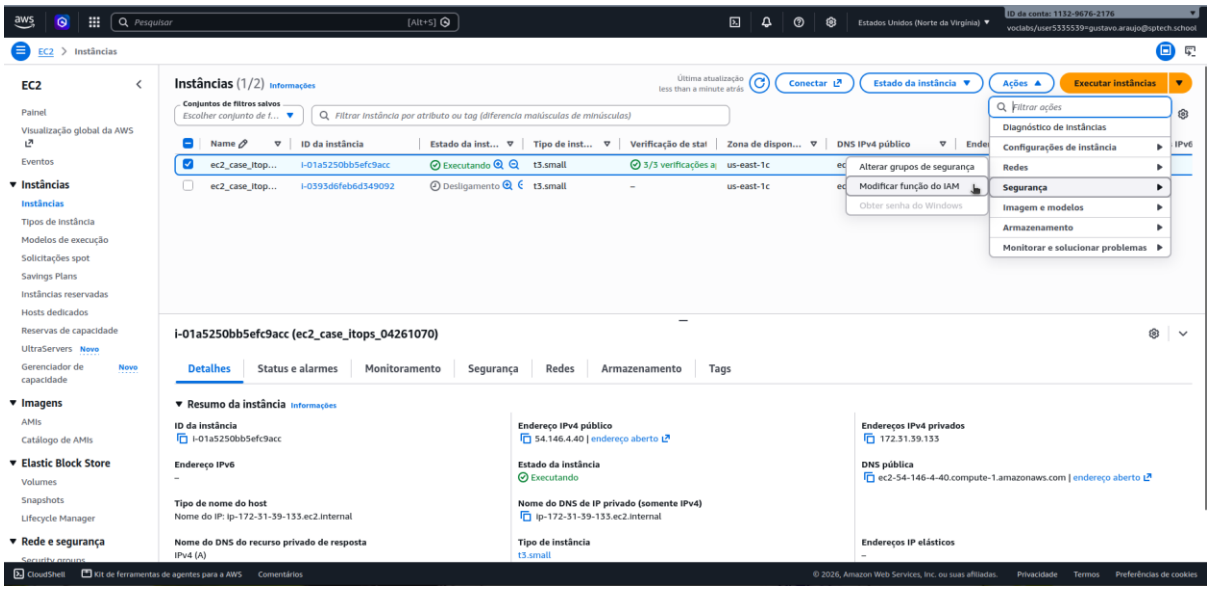
4. Criando uma Instância na EC2



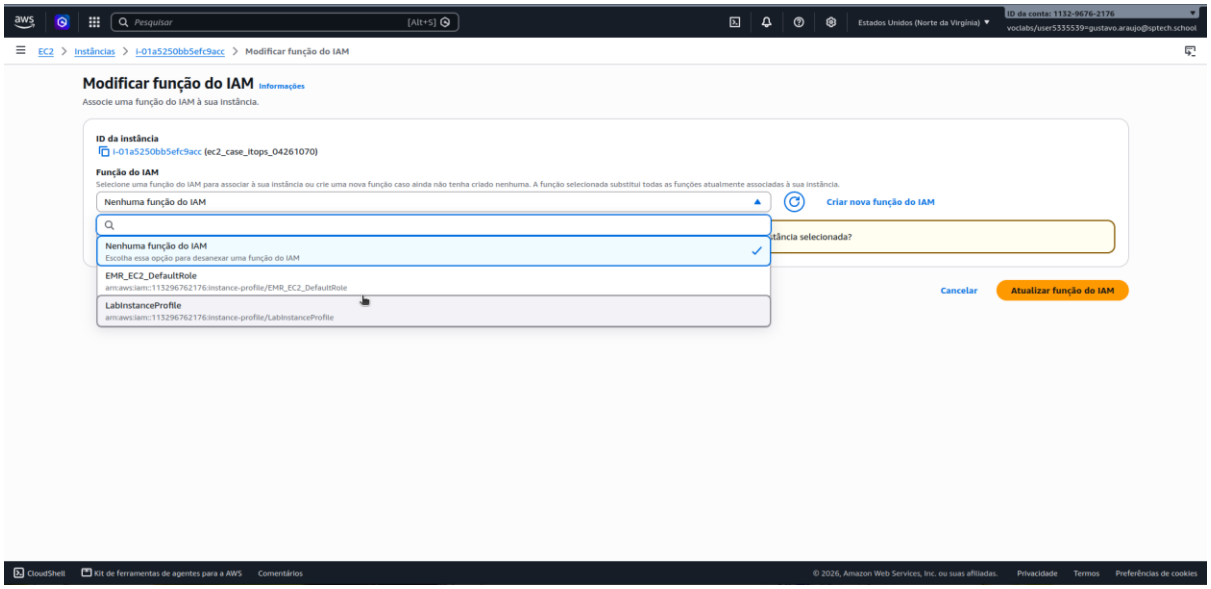
Criou uma instância utilizando o recurso da EC2 para resolver o armazenar os meus arquivos python representando as ETLs. Utilizando o sistema operacional Ubuntu 26.04, além do tipo de servidor sendo o t3.small(com 2 vCPUs e 2 GiB de memória RAM) e 8Gib de armazenamento em Disco. Além de ser chamada de “ec2_case_itops_04261070”.



No terminal local, preparo a chave de acesso SSH (.pem) para conectar à instância EC2. Primeiro, movo o arquivo do diretório de Downloads para a pasta atual e ajusto suas permissões de segurança utilizando o comando chmod 400. Em seguida, realizo o acesso remoto à instância Ubuntu executando o comando ssh -i referenciando a chave e o endereço DNS público da máquina, obtendo sucesso na conexão.



Retorno ao console da AWS, na seção de instâncias do serviço EC2, para configurar as permissões da máquina virtual que está em execução. Seleciono a instância do projeto, abro o menu "Ações", navego até a opção "Segurança" e clico em "Modificar função do IAM".

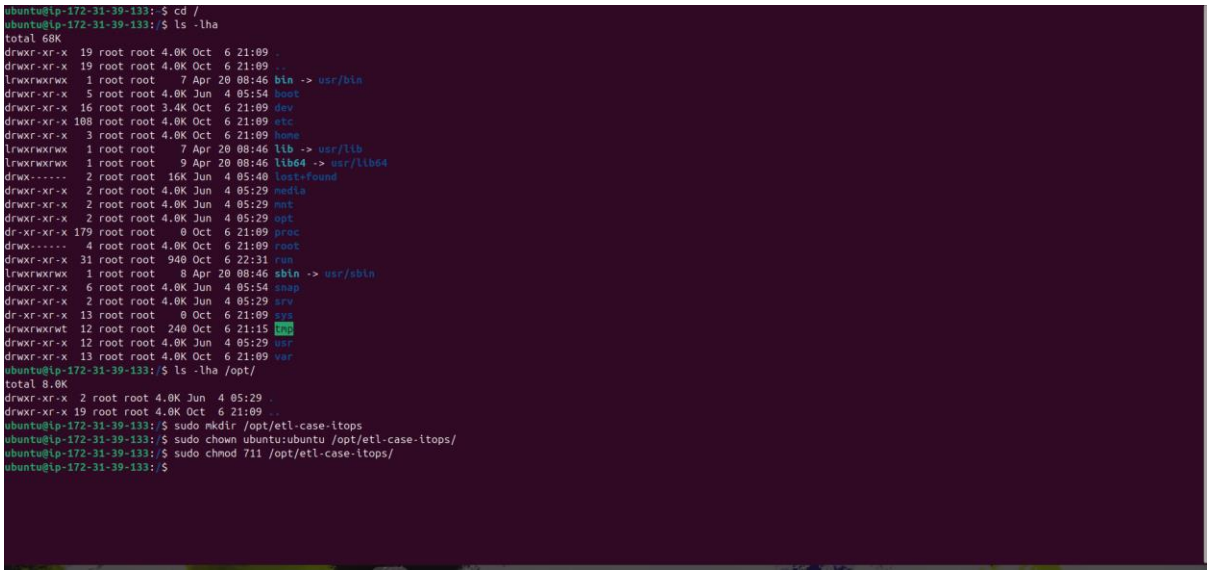


Na tela de modificação da função do IAM da instância selecionada, clico no menu suspenso para escolher o perfil de acesso. Seleciono a opção LabInstanceProfile, que concederá à EC2 os privilégios e permissões necessárias para interagir de forma segura com outros recursos da AWS.

```
eee_W_6651586@172.31.4.181:~$ aws s3 ls
2026-10-06 14:10:41 bucket-case-itops-04261070
eee_W_6651586@172.31.4.181:~$
```

Para validar as permissões de leitura do ambiente, executo o comando de listagem aws s3 ls através do terminal. O comando retorna a listagem contendo o bucket bucket-case-itops-04261070 e sua data de criação, confirmando que a comunicação de leitura com o S3 está funcionando corretamente

5. Configurando instância para receber arquivos python



Dentro do acesso SSH da instância EC2, inicio a preparação do sistema de arquivos para receber os scripts do processo de ETL. Utilizo o comando sudo mkdir para criar o diretório /opt/etl-case-itops. Em seguida, altero o proprietário desta nova pasta para o usuário da máquina com o comando sudo chown ubuntu:ubuntu e ajusto as permissões de execução com sudo chmod 711.

6. Arquivos Python

6.1. captura-access-point.py

```
import psutil
import json
from datetime import datetime
import random
import io, boto3
import os
from dotenv import load_dotenv

dados = []
load_dotenv()
```

Trago as bibliotecas necessárias para o script funcionar. O destaque aqui é o psutil (usado para ler dados de hardware do access pointer e firewall, como CPU e memória) e o random (para gerar números aleatórios). Também prepara uma lista vazia dados para guardar as informações.

```
session = boto3.Session(
aws_access_key_id=os.getenv("AWS_ACCESS_KEY_ID"),
aws_secret_access_key=os.getenv("AWS_SECRET_ACCESS_KEY"),
aws_session_token=os.getenv("AWS_SESSION_TOKEN"),
region_name="us-east-1",
)
s3_client = session.client("s3")
```

Uso as variáveis do .env para fazer login na Amazon Web Services (AWS) e cria uma conexão com o serviço S3.

```
try:
    id_antena = int(input('Digite o identificador da antena: '))
    contador = 0

    acess_point_json = {
        "ID_antena":0,
        "Bytes_Sent":0,
        "Bytes_Recv":0,
        "Active_conn":0,
        "CPU_Usage":0,
        "RAM_Usage":0
    }
```

Utilizo a função input() para ter uma interatividade com o usuário e para quem rodar o

script digitar o número (ID) do Access Point (Antena). Depois, cria um "molde" (dicionário) vazio com a estrutura dos dados que serão coletados (Bytes, conexões, CPU, RAM) e inicia um contador.

```
while True:
        json_ap = acess_point_json.copy()

        numero_medio_dispositivos = 100

        variacao_dispositivos = random.randint(1, 20)
        chance_variar = random.randint(1, 2)

        json_ap["ID_antena"] = id_antena
        json_ap["Bytes_Sent"] = psutil.net_io_counters().bytes_sent
        json_ap["Bytes_Recv"] = psutil.net_io_counters().bytes_recv

        if(chance_variar == 2):
            json_ap["Active_conn"] = numero_medio_dispositivos - variacao_dispositivos
        else:
            json_ap["Active_conn"] = numero_medio_dispositivos

        json_ap["CPU_Usage"] = psutil.cpu_percent(interval=1)

        memoria_ram = psutil.virtual_memory()

        memoria_ram_total = memoria_ram.total
        memoria_ram_disponivel = memoria_ram.available

        porcentagem_ram = ((memoria_ram_total - memoria_ram_disponivel) / memoria_ram_total * 100)

        json_ap["RAM_Usage"] = porcentagem_ram

        print(f"Capturando dados...\nID_antena: {id_antena}\nBytes_Sent: {json_ap["Bytes_Sent"]}\nBytes_Recv: {json_ap["Bytes_Recv"]}
        \nCPU_Usage: {json_ap["CPU_Usage"]}\nRAM_Usage: {json_ap["RAM_Usage"]}\n\n")

        dados.append(json_ap.copy())

        contador += 1
```

Inicio um loop infinito (while True). A cada rodada, simulo uma quantidade de dispositivos conectados usando números aleatórios, usa o psutil para capturar as informações reais da máquina onde o código está rodando (tráfego de rede, uso de CPU e uso de RAM) e, por fim, mostro esses dados na tela, guarda na lista dados e aumenta o contador em 1. O cpu_percent(interval=1) faz com que cada coleta dure 1 segundo.

```
if(contador == 60):
    print("Dados capturados!\n\n")
    contador = 0

    nome_arquivo = f"{datetime.now().strftime('%Y-%m-%d_%H-%M')}_ap{id_antena}.json"

    s3_client.put_object(
        Bucket=os.getenv("NOME_BUCKET"),
        Key=f"01-bronze/{nome_arquivo}",
        Body=json.dumps(dados, ensure_ascii=False, indent=4).encode('utf-8')
    )
```

Verifico se o contador chegou a 60 (ou seja, após ~60 segundos de coleta). Se chegou, ele zera o contador, empacota todas as 60 leituras em um arquivo .json com a data/hora no nome e faz o upload direto para a pasta 01-bronze na nuvem (AWS S3). Depois, o loop continua coletando mais dados.

```
except ValueError:
    print("Insira um número válido!")
```

Garanto que o programa não "quebre" feio caso o usuário digite uma letra ou palavra lá no começo, quando o código pede o número identificador da antena.

6.2. captura-firewall.py

```
import psutil
import json
from datetime import datetime
import random
import io, boto3
import os
from dotenv import load_dotenv

dados = []
load_dotenv()

session = boto3.Session(
aws_access_key_id=os.getenv("AWS_ACCESS_KEY_ID"),
aws_secret_access_key=os.getenv("AWS_SECRET_ACCESS_KEY"),
aws_session_token=os.getenv("AWS_SESSION_TOKEN"),
region_name="us-east-1",
)
s3_client = session.client("s3")
```

Trago para o código as ferramentas necessárias para trabalhar com datas, ler arquivos na nuvem, gerar planilhas (CSV) e lidar com variáveis de ambiente (onde ficam guardadas senhas e chaves secretas com segurança). Além de usar as variáveis do .env para fazer login na Amazon Web Services (AWS) e cria uma conexão com o serviço S3.

```
contador = 0

firewall_json = {
    "Active_sessions":0,
    "Dropped_packets":0,
    "top_blocked_ip":0,
    "CPU_Usage":0,
    "RAM_Usage":0,
    "Bytes_Sent":0,
    "Bytes_Recv":0,
}
```

Crio um "molde" vazio para os dados. A diferença aqui é que as métricas são típicas de segurança de rede, como "Sessões ativas", "Pacotes derrubados/perdidos" e o "IP mais bloqueado".

```
while True:
        json_ap = firewall_json.copy()

        numero_medio_sessoes = 100

        variacao_dispositivos = random.randint(1, 20)
        chance_variar = random.randint(1, 2)

        numero_pacote_perdidos = 0

        pacotes_perdidos = random.randint(0, 100)
        chance_perder = random.randint(1, 5)

        if (chance_variar == 2):
            json_ap["Active_sessions"] = numero_medio_sessoes - variacao_dispositivos
        else:
            json_ap["Active_sessions"] = numero_medio_sessoes

        if (chance_perder == 5):
            json_ap["Dropped_packets"] = numero_pacote_perdidos
        else:
            json_ap["Dropped_packets"] = numero_pacote_perdidos + pacotes_perdidos

        ip = ""

        for i in range(4):
            ip_random = random.randint(0, 255)

            if(i == 3):
                ip += f'{ip_random}'
            else:
                ip += f'{ip_random}.'

        json_ap["top_blocked_ip"] = ip
```

Inicio o ciclo infinito de coleta (1 por segundo). Aqui, ele usa muita matemática aleatória (random) para inventar dados de segurança. Ele cria pequenas variações de conexões ativas, simula uma quantidade de "pacotes derrubados" (ataques barrados) e gera um endereço de IP falso e aleatório (como 172.45.12.9) para simular qual IP foi mais bloqueado naquele segundo.

```
json_ap["CPU_Usage"] = psutil.cpu_percent(interval=1)

memoria_ram = psutil.virtual_memory()

memoria_ram_total = memoria_ram.total
memoria_ram_disponivel = memoria_ram.available

porcentagem_ram = ((memoria_ram_total - memoria_ram_disponivel) / memoria_ram_total * 100)

json_ap["RAM_Usage"] = porcentagem_ram

aleatorizando_rede_sent = random.randint(0, 30)
aleatorizando_rede_recv = random.randint(0, 50)

json_ap["Bytes_Sent"] = (psutil.net_io_counters().bytes_sent) * aleatorizando_rede_sent
json_ap["Bytes_Recv"] = (psutil.net_io_counters().bytes_recv) * aleatorizando_rede_recv

print(f"Capturando dados...\nActive_sessions: {json_ap["Active_sessions"]}\nDropped_packets: {json_ap["Dropped_packets"]}
\ntop_blocked_ip: {ip}\nCPU_Usage: {json_ap["CPU_Usage"]}\nRAM_Usage: {json_ap["RAM_Usage"]}\nBytes_Sent: {json_ap["Bytes_Sent"]}
\nBytes_Recv: {json_ap["Bytes_Recv"]}\n\n")

dados.append(json_ap.copy())

contador += 1
```

Leio o uso real de CPU e Memória RAM do seu computador. A grande diferença para o código anterior está na Rede: como um Firewall de verdade processa muito mais dados que uma simples antena, o código pega o tráfego real da sua máquina e multiplica por um fator aleatório (de 0 a 30 ou 0 a 50) para simular um volume de dados muito maior.

```
if(contador == 60):
    print("Dados capturados!\n\n")
    contador = 0

    nome_arquivo = f"{datetime.now().strftime('%Y-%m-%d_%H-%M')}_firewall.json"

    s3_client.put_object(
        Bucket=os.getenv("NOME_BUCKET"),
        Key=f"01-bronze/{nome_arquivo}",
        Body=json.dumps(dados, ensure_ascii=False, indent=4).encode('utf-8')
    )
```

A cada 60 leituras (cerca de 1 minuto), ele salva tudo em um arquivo .json (desta vez com _firewall no nome) e faz o upload desse arquivo para a pasta 01-bronze no S3 da Amazon.

6.3. bronze2silver.py

```python
import os
import json
import csv
import boto3
from datetime import datetime, timedelta
from dotenv import load_dotenv

load_dotenv()
```

Trago para o código as ferramentas necessárias para trabalhar com datas, ler arquivos na nuvem, gerar planilhas (CSV) e lidar com variáveis de ambiente (onde ficam guardadas senhas e chaves secretas com segurança).

```python
session = boto3.Session(
    aws_access_key_id=os.getenv("AWS_ACCESS_KEY_ID"),
    aws_secret_access_key=os.getenv("AWS_SECRET_ACCESS_KEY"),
    aws_session_token=os.getenv("AWS_SESSION_TOKEN"),
    region_name="us-east-1",
)
s3_client = session.client("s3")
```

Uso as variáveis do .env para fazer login na Amazon Web Services (AWS) e cria uma conexão com o serviço S3.

```python
resposta = s3_client.list_objects_v2(Bucket=os.getenv("NOME_BUCKET"), Prefix="01-bronze")
arquivos_no_bucket = resposta.get('Contents', [])

agora = datetime.now()

dados_ap = []
dados_firewall = []
```

Vou até a pasta 01-bronze no S3 e lista todos os arquivos que estão lá dentro. Também crio as listas vazias que vão armazenar os dados de Access Point e Firewall mais para frente.

```
for obj in arquivos_no_bucket:
    chave_arquivo = obj['Key']
    nome_arquivo = chave_arquivo.split('/')[-1]

    if nome_arquivo.endswith(".json"):
        try:
            str_data_arquivo = nome_arquivo[:16]
            data_arquivo = datetime.strptime(str_data_arquivo, '%Y-%m-%d_%H-%M')
        except ValueError:
            continue

        diferenca = agora - data_arquivo

        if timedelta(minutes=0) <= diferenca <= timedelta(minutes=3, seconds=59):
            resposta_s3 = s3_client.get_object(Bucket=os.getenv("NOME_BUCKET"), Key=chave_arquivo)
            conteudo_texto = resposta_s3['Body'].read().decode('utf-8')
            dados_json = json.loads(conteudo_texto)
```

Leio o nome de cada arquivo .json para descobrir a data e hora em que ele foi gerado. Ele filtra e pega apenas os arquivos que foram criados nos últimos 4 minutos (entre 0 e 3m59s).

```
if "firewall" in nome_arquivo.lower():
    dados_firewall.extend(dados_json if isinstance(dados_json, list) else [dados_json])
elif "ap" in nome_arquivo.lower():
    dados_ap.extend(dados_json if isinstance(dados_json, list) else [dados_json])
```

Abro e leio os arquivos recentes que passaram no filtro anterior. Se o nome do arquivo tiver "firewall", ele guarda na lista de firewall; se tiver "ap", guarda na lista de Access Point.

```
if not dados_ap or not dados_firewall:
    print("Nenhum arquivo válido encontrado dentro do prazo de 3m59s em dados_ap ou dados_firewall.")
    exit()

mudancas_cabecalho_access_point = {
    "Bytes_Sent": "Bytes_Sent_AP",
    "Bytes_Recv": "Bytes_Recv_AP",
    "CPU_Usage": "CPU_Usage_AP",
    "RAM_Usage": "RAM_Usage_AP"
}

mudancas_cabecalho_firewall = {
    "CPU_Usage": "CPU_Usage_Firewall",
    "RAM_Usage": "RAM_Usage_Firewall",
    "Bytes_Sent": "Bytes_Sent_Firewall",
    "Bytes_Recv": "Bytes_Recv_Firewall"
}

cabecalho = ["timestamp"]

for item in list(dados_ap[0].keys()):
    cabecalho.append(mudancas_cabecalho_access_point.get(item, item))

for item in list(dados_firewall[0].keys()):
    cabecalho.append(mudancas_cabecalho_firewall.get(item, item))

cabecalho.extend([
    "alta_densidade",
    "gargalo_processamento_ap",
    "gargalo_processamento_firewall",
    "out_of_memory_ap",
    "out_of_memory_firewall",
    "consistência_trafego"
])

dados = []

limite_linhas = min(len(dados_ap), len(dados_firewall))
```

Preparo os títulos das colunas para a nova planilha que será gerada. Renomeio dados que têm o mesmo nome (como "CPU_Usage") para sabermos o que é CPU do Firewall e o que

é CPU do AP. Também adiciono colunas novas para diagnósticos (como "alta_densidade", "out_of_memory", etc).

```
for i in range(limite_linhas):
    data_hora_atual = datetime.now()
    data_hora_formatada = data_hora_atual.strftime('%Y-%m-%d %H:%M:%S')

    alta_densidade = "Sim" if dados_ap[i].get("Active_conn", 0) > 40 else "Não"
    gargalo_processamento_ap = "Sim" if dados_ap[i].get("CPU_Usage", 0) > 80 else "Não"
    gargalo_processamento_firewall = "Sim" if dados_firewall[i].get("CPU_Usage", 0) > 80 else "Não"
    out_of_memory_ap = "Sim" if dados_ap[i].get("RAM_Usage", 0) > 75 else "Não"
    out_of_memory_firewall = "Sim" if dados_firewall[i].get("RAM_Usage", 0) > 75 else "Não"

    bytes_fw = dados_firewall[i].get("Bytes_Sent", 0)
    bytes_ap = dados_ap[i].get("Bytes_Sent", 0)

    if bytes_fw > bytes_ap:
        consistencia_trafego = "Firewall Enviando mais que recebendo"
    elif bytes_fw < bytes_ap:
        consistencia_trafego = "Access Point enviando mais que firewall recebendo"
    else:
        consistencia_trafego = "Firewall e Access Point em sincrônia"

    linha = [
        data_hora_formatada,
        *dados_ap[i].values(),
        *dados_firewall[i].values(),
        alta_densidade,
        gargalo_processamento_ap,
        gargalo_processamento_firewall,
        out_of_memory_ap,
        out_of_memory_firewall,
        consistencia_trafego
    ]
    dados.append(linha)
```

Junto os dados do AP e do Firewall. Aqui o código aplica "regras de negócio" (inteligência): ele checa se tem conexões demais, se o uso de CPU passou de 80% ou a RAM passou de 75%, e compara quem está enviando/recebendo mais dados. Tudo isso vira uma linha de diagnóstico na nossa planilha.

```
with open(nome_arquivo_saida, "w", newline="", encoding="utf-8") as csvFile:
    escritor = csv.writer(csvFile, delimiter=';')
    escritor.writerow(cabecalho)
    escritor.writerows(dados)

s3_client.upload_file(
    Filename=nome_arquivo_saida,
    Bucket=os.getenv("NOME_BUCKET"),
    Key=f"02-silver/{nome_arquivo_saida}"
)

print(f"Relatório enviado com sucesso: 02-silver/{nome_arquivo_saida}")
```

Salvo todas essas análises e cruzamentos num arquivo .csv no computador e, em seguida, faz o envio (upload) desse arquivo finalizado de volta para a nuvem da AWS, mas agora numa pasta de dados processados chamada 02-silver.

6.4. silver2gold.py

```python
import os
import csv
from datetime import datetime
import boto3
from datetime import datetime, timedelta
from dotenv import load_dotenv

load_dotenv()

session = boto3.Session(
aws_access_key_id=os.getenv("AWS_ACCESS_KEY_ID"),
aws_secret_access_key=os.getenv("AWS_SECRET_ACCESS_KEY"),
aws_session_token=os.getenv("AWS_SESSION_TOKEN"),
region_name="us-east-1",
)
s3_client = session.client("s3")
```

Trago para o código as ferramentas necessárias para trabalhar com datas, ler arquivos na nuvem, gerar planilhas (CSV) e lidar com variáveis de ambiente (onde ficam guardadas senhas e chaves secretas com segurança). Além de usar as variáveis do .env para fazer login na Amazon Web Services (AWS) e cria uma conexão com o serviço S3.

```python
resposta = s3_client.list_objects_v2(Bucket=os.getenv("NOME_BUCKET"), Prefix="02-silver")
arquivos_no_bucket = resposta.get('Contents', [])

agora = datetime.now()
linhas_silver = []

for obj in arquivos_no_bucket:
    chave_arquivo = obj['Key']
    nome_arquivo = chave_arquivo.split('/')[-1]

    if ".csv" in nome_arquivo and "bronze2silver" in nome_arquivo:

        str_data_arquivo = nome_arquivo[:16]


        data_arquivo = datetime.strptime(str_data_arquivo, '%Y-%m-%d_%H-%M')
        diferenca = agora - data_arquivo

        if timedelta(minutes=0) <= diferenca <= timedelta(minutes=3, seconds=59):
            print(f"Lendo CSV válido dentro do prazo: {nome_arquivo}")

            resposta_s3 = s3_client.get_object(Bucket=os.getenv("NOME_BUCKET"), Key=chave_arquivo)

            conteudo_texto = resposta_s3['Body'].read().decode('utf-8')

            linhas = conteudo_texto.splitlines()

            if len(linhas) > 0:
                cabecalho_original = linhas[0].split(';')

                for linha_texto in linhas[1:]:
                    if linha_texto.strip():
                        linha = linha_texto.split(';')
                        linhas_silver.append(linha)

agrupado_antenas = {}
```

Vou na pasta 02-silver na nuvem e procura o arquivo .csv que foi gerado pelo arquivo python bronze2silver que tenha sido criado nos últimos 4 minutos. Abro esse arquivo e leio todas as linhas para começar a trabalhar.

```python
for linha in linhas_silver:
    id_antena = linha[1]
    bytes_sent_ap = float(linha[2])
    bytes_recv_ap = float(linha[3])
    active_conn = float(linha[4])
    cpu_ap = float(linha[5])
    ram_ap = float(linha[6])

    dropped_packets = float(linha[8])
    cpu_firewall = float(linha[10])
    bytes_sent_fw = float(linha[12])
    bytes_recv_fw = float(linha[13])

    if id_antena not in agrupado_antenas:
        agrupado_antenas[id_antena] = {
            "total_registros": 0,
            "soma_bytes_sent_ap": 0.0,
            "soma_bytes_recv_ap": 0.0,
            "soma_active_conn": 0.0,
            "soma_cpu_ap": 0.0,
            "soma_ram_ap": 0.0,
            "soma_dropped_packets": 0.0,
            "soma_cpu_firewall": 0.0,
            "soma_bytes_sent_fw": 0.0,
            "soma_bytes_recv_fw": 0.0,
            "ultimas_conexoes": []
        }

    agrupado_antenas[id_antena]["total_registros"] += 1
    agrupado_antenas[id_antena]["soma_bytes_sent_ap"] += bytes_sent_ap
    agrupado_antenas[id_antena]["soma_bytes_recv_ap"] += bytes_recv_ap
    agrupado_antenas[id_antena]["soma_active_conn"] += active_conn
    agrupado_antenas[id_antena]["soma_cpu_ap"] += cpu_ap
    agrupado_antenas[id_antena]["soma_ram_ap"] += ram_ap
    agrupado_antenas[id_antena]["soma_dropped_packets"] += dropped_packets
    agrupado_antenas[id_antena]["soma_cpu_firewall"] += cpu_firewall
    agrupado_antenas[id_antena]["soma_bytes_sent_fw"] += bytes_sent_fw
    agrupado_antenas[id_antena]["soma_bytes_recv_fw"] += bytes_recv_fw
    agrupado_antenas[id_antena]["ultimas_conexoes"].append(active_conn)
```

Como a planilha "Silver" tem várias linhas misturadas, este bloco organiza tudo. Ele cria um grupo para cada ID_antena e começa a somar todo o tráfego, pacotes e conexões daquela antena específica. Isso é necessário para tirar médias depois.

```python
trafego_total_predio = 0.0
for id_antena in agrupado_antenas:
    trafego_total_predio += agrupado_antenas[id_antena]["soma_bytes_sent_ap"] + agrupado_antenas[id_antena]["soma_bytes_recv_ap"]
```

Somo o tráfego de todas as antenas juntas para descobrir o quanto a rede inteira (o "prédio") está consumindo.

```
cabecalho = [
    "ID_antena",
    "total_trafego_ap_bytes",
    "media_conexoes_ativas",
    "media_cpu_ap",
    "eficiencia_hardware_ratio",
    "percentual_consumo_predio",
    "zona_subutilizada",
    "predicao_sobrecarga_risco",
    "analise_expurgo_seguranca",
    "gargalo_saida_firewall"
]

dados = []

limite_conexoes_ap = 50.0

for id_antena in agrupado_antenas:
    total_reg = agrupado_antenas[id_antena]["total_registros"]

    media_conn = agrupado_antenas[id_antena]["soma_active_conn"] / total_reg
    media_cpu_ap = agrupado_antenas[id_antena]["soma_cpu_ap"] / total_reg
    media_cpu_fw = agrupado_antenas[id_antena]["soma_cpu_firewall"] / total_reg

    total_bytes_ap = agrupado_antenas[id_antena]["soma_bytes_sent_ap"] + agrupado_antenas[id_antena]["soma_bytes_recv_ap"]

    soma_todas_conexoes = 0.0
    quantidade_antenas = len(agrupado_antenas)

    for info in agrupado_antenas.values():
        soma_todas_conexoes += (info["soma_active_conn"] / info["total_registros"])

    media_global_conexoes = soma_todas_conexoes / quantidade_antenas
    media_global_trafego = trafego_total_predio / quantidade_antenas

    limite_conexoes_predio = media_global_conexoes
    limite_trafego_predio = media_global_trafego

    if (media_conn < limite_conexoes_predio and total_bytes_ap < limite_trafego_predio):
        zona_subutilizada = "Sim - Antena com uso muito abaixo da média do prédio (Sugerido Redimensionar)"
    else:
        zona_subutilizada = "Não - Uso normal ou elevado"
```

```
    if media_cpu_ap > 0:
        eficiencia_hardware = total_bytes_ap / media_cpu_ap
    else:
        eficiencia_hardware = total_bytes_ap

    conexoes = agrupado_antenas[id_antena]["ultimas_conexoes"]
    if len(conexoes) >= 2:
        tendencia_crescimento = conexoes[-1] - conexoes[0]
    else:
        tendencia_crescimento = 0

    if media_conn >= limite_conexoes_ap * 0.8:
        predicao_sobrecarga_risco = "ALTO - AP perto do limite de conexões simultâneas"
    elif tendencia_crescimento > 10:
        predicao_sobrecarga_risco = "MÉDIO - Tendência de alta no número de conexões"
    else:
        predicao_sobrecarga_risco = "BAIXO - Operação estável"

    if trafego_total_predio > 0:
        percentual_consumo = round((total_bytes_ap / trafego_total_predio) * 100, 2)
    else:
        percentual_consumo = 0.0

    total_pacotes_bloqueados = agrupado_antenas[id_antena]["soma_dropped_packets"]
    if total_pacotes_bloqueados > 500 and media_cpu_fw > 75:
        analise_expurgo_seguranca = "CRÍTICO - Alto volume de descartes e CPU do Firewall saturada (Possível DDoS)"
    elif total_pacotes_bloqueados > 200:
        analise_expurgo_seguranca = "ALERTA - Volume elevado de pacotes bloqueados"
    else:
        analise_expurgo_seguranca = "NORMAL - Descarte de pacotes sob controle"

    if media_cpu_fw > 80:
        gargalo_saida_firewall = "Gargalo no Hardware do Firewall (Processamento de Regras esgotado)"
    else:
        gargalo_saida_firewall = "Firewall operando dentro da margem de segurança"

    dados.append([id_antena, total_bytes_ap, round(media_conn, 2), round(media_cpu_ap, 2), round(eficiencia_hardware, 2), {percentual_consumo},
    zona_subutilizada, predicao_sobrecarga_risco, analise_expurgo_seguranca, gargalo_saida_firewall])
```

Esta é a parte mais inteligente do código. Usando as médias calculadas, ele tira conclusões de alto nível, descobrindo se alguma antena está "sobrando" (sendo pouco usada) comparada ao resto do prédio. Além de tentar prever se uma antena vai cair em breve analisando se o número de conexões está crescendo rápido. Por fim, ele cruza os dados da Antena e do Firewall para detectar possíveis ataques hackers (DDoS). Se tiver muitos pacotes bloqueados e a CPU do firewall estiver no talo, ele soa um alerta "CRÍTICO".

```
nome_arquivo_saida = f"{datetime.now().strftime('%Y-%m-%d_%H-%M')}_silver2gold.csv"

with open(nome_arquivo_saida, "w", newline="", encoding="utf-8") as csvFile:
    csv.writer(csvFile, delimiter=';').writerow(cabecalho)

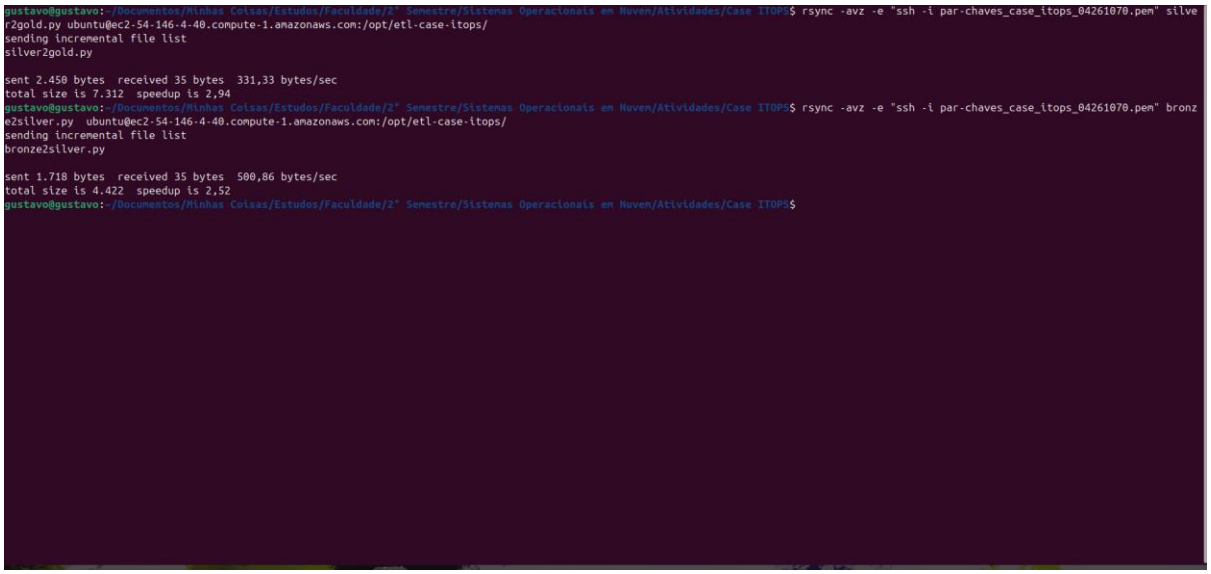
    for i in dados:
        csv.writer(csvFile, delimiter=';').writerow(i)

s3_client.upload_file(
    Filename=nome_arquivo_saida,
    Bucket=os.getenv("NOME_BUCKET"),
    Key=f"03-gold/{nome_arquivo_saida}"
)

print(f"Relatório de negócio gerado com sucesso: {nome_arquivo_saida}")
```

Salvo todos esses diagnósticos avançados em um novo arquivo .csv (chamado _silver2gold) e o envia para a pasta final, a 03-gold na nuvem da AWS.

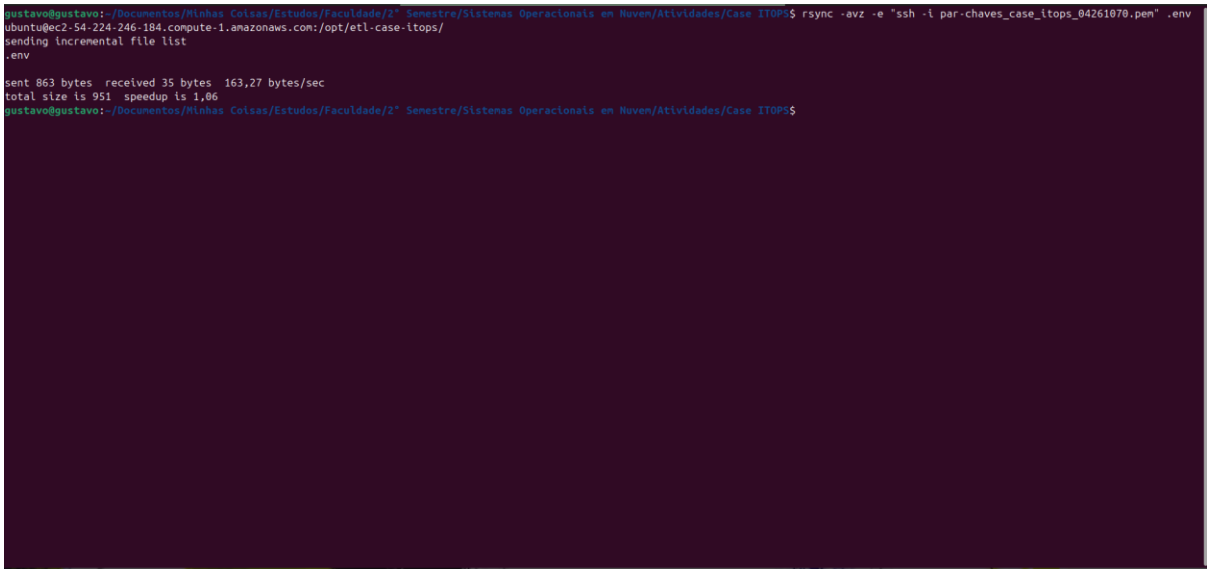
7. Enviando os arquivos python e .env para a instância



A partir do meu terminal local, utilizo a ferramenta rsync combinada com a chave SSH para realizar a transferência dos códigos Python para o servidor EC2. Executo o comando para enviar primeiramente o script silver2gold.py para o diretório /opt/etl-case-itops/ remoto e, logo na sequência, realizo o mesmo procedimento para enviar o arquivo bronze2silver.py



De volta ao terminal remoto da instância EC2, executo o comando ls -lha /opt/etl-case-itops/ para listar o conteúdo e as propriedades do diretório. Confirmo visualmente que os arquivos bronze2silver.py e silver2gold.py foram recebidos com sucesso e já estão devidamente alocados com a propriedade do usuário "ubuntu".

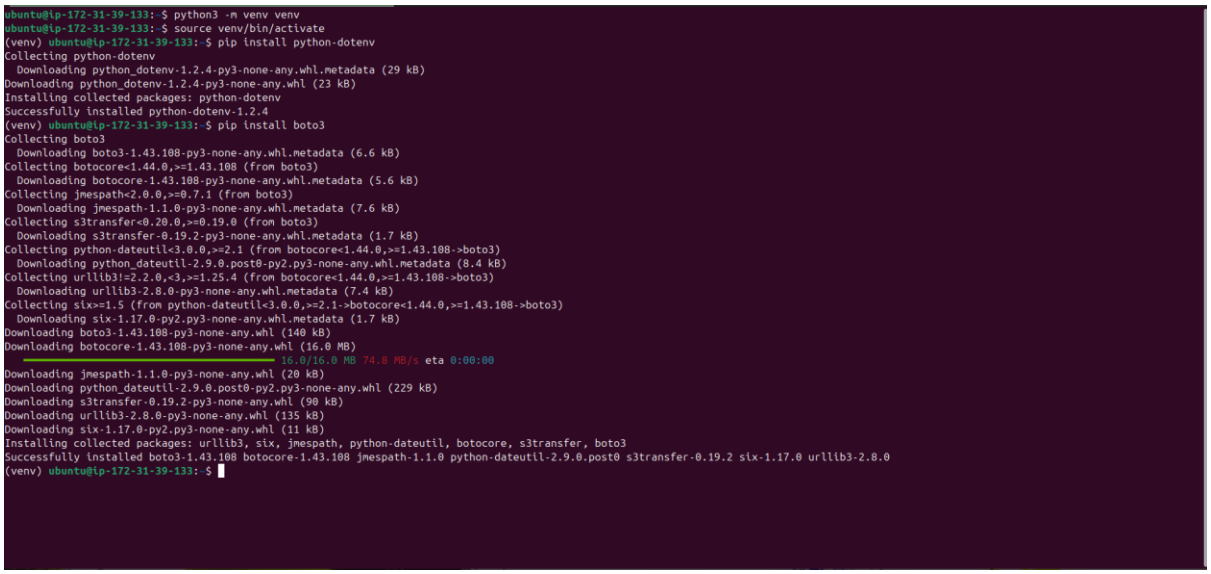


Retorno ao terminal local para realizar uma última transferência utilizando novamente o utilitário rsync com a chave .pem. Desta vez, executo o comando para enviar o arquivo oculto de variáveis de ambiente (.env) para o diretório /opt/etl-case-itops/ na instância, garantindo os parâmetros de configuração da aplicação.



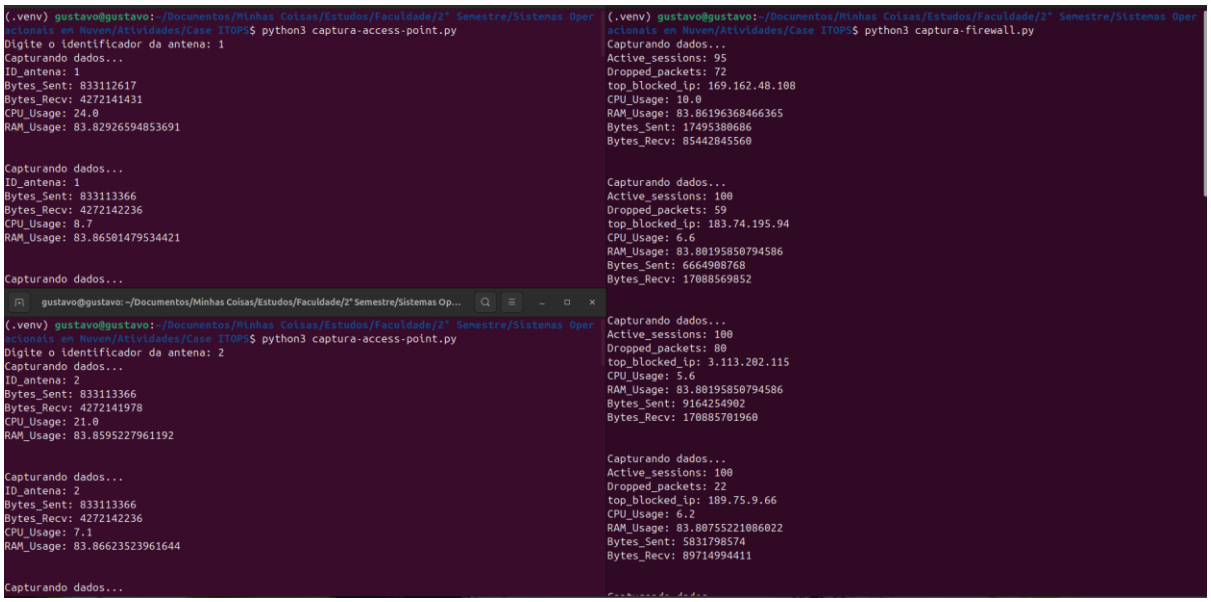
Finalmente, executo uma nova listagem com o comando ls -lha no diretório /opt/etl-case-itops/ através do acesso SSH da instância. Valido que os três arquivos indispensáveis (.env, bronze2silver.py e silver2gold.py) estão armazenados corretamente na pasta, concluindo a etapa de preparação dos artefatos de código no servidor remoto.

8. Configurando python e venv na instância

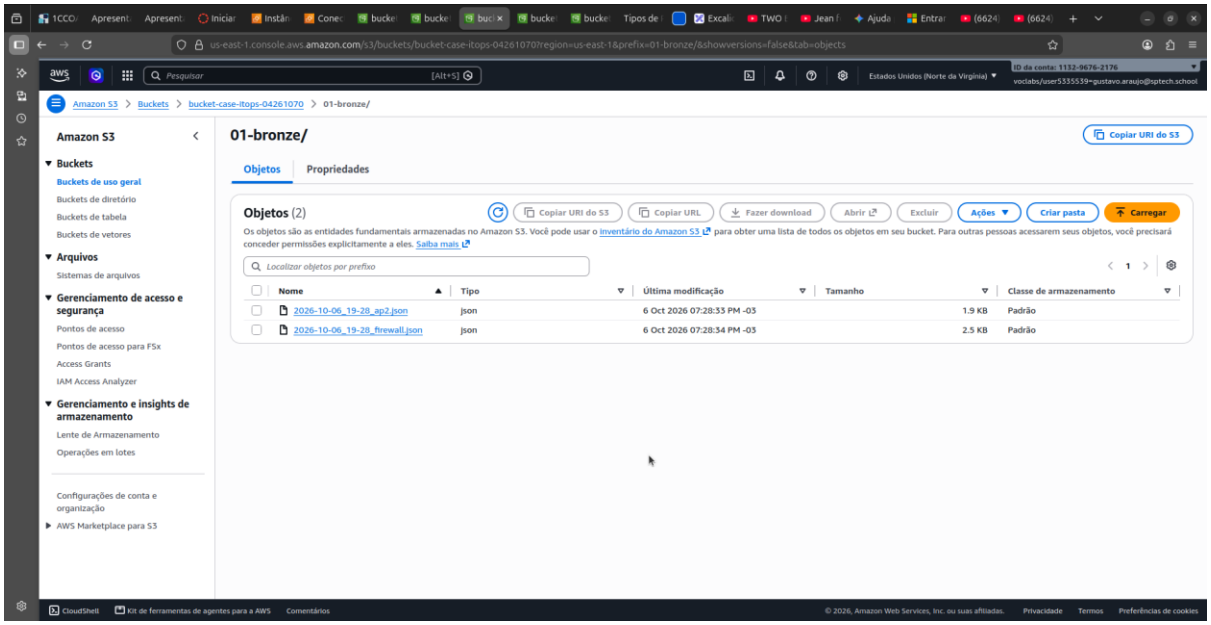


No terminal da instância EC2, realizo a configuração do ambiente Python isolado para a execução segura dos scripts. Primeiro, crio e ativo um ambiente virtual utilizando os comandos python3 -m venv venv e source venv/bin/activate. Na sequência, executo o gerenciador de pacotes pip para instalar as bibliotecas essenciais da aplicação, incluindo o python-dotenv e o boto3 para integração com os serviços da AWS.

9. Inserindo dados no Bucket

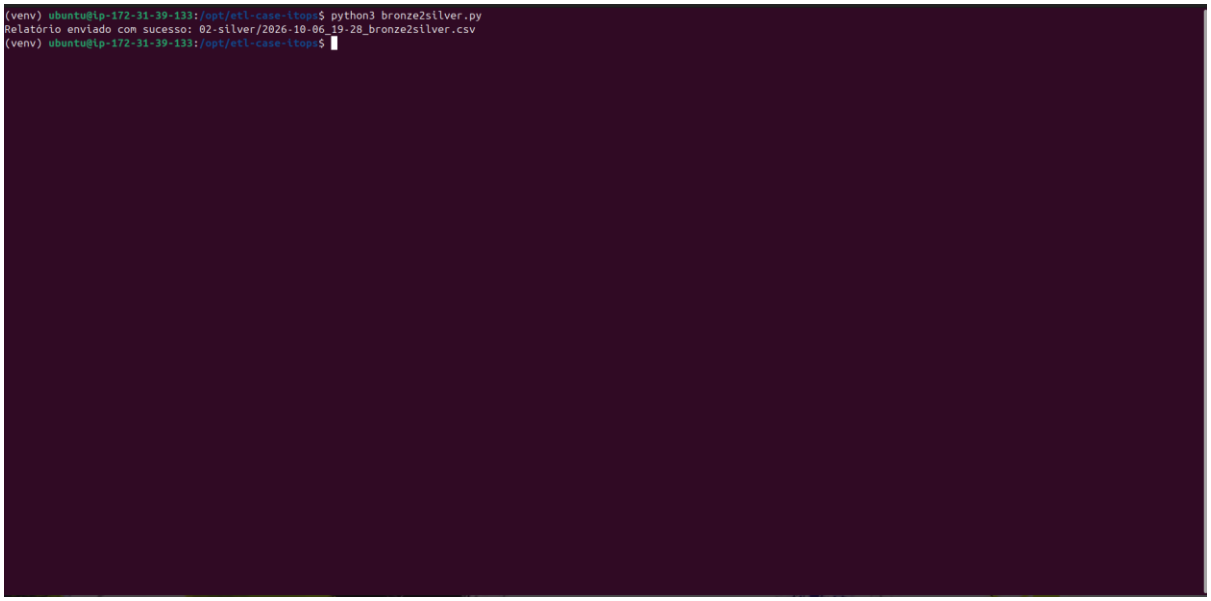


Em meu ambiente local, organizo múltiplos terminais simultâneos para simular o monitoramento e a geração de logs da infraestrutura. Inicializo instâncias do script captura-access-point.py para as antenas 1 e 2, além do script captura-firewall.py, gerando uma captura contínua de dados de consumo de CPU, RAM, pacotes bloqueados e uso de rede que irão alimentar o fluxo do Data Lake.

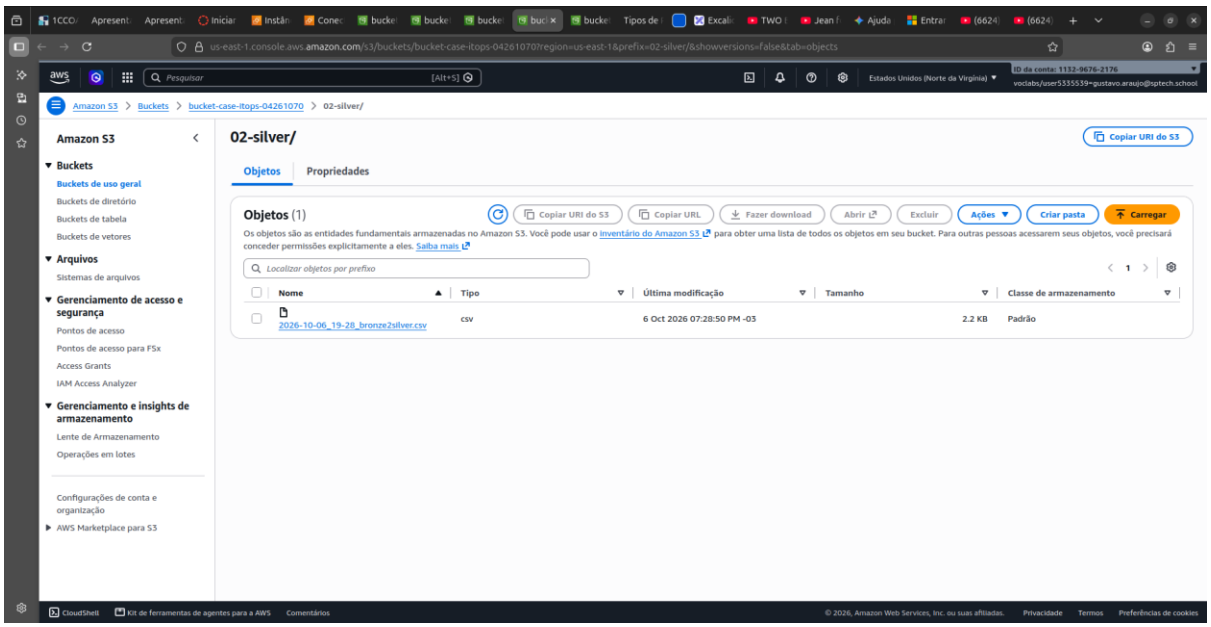


Acesso a interface do Amazon S3 para validar a ingestão dos dados brutos gerados pela simulação. Ao acessar a pasta 01-bronze/ do bucket-case-itops-04261070, confirmo que os arquivos com formato .json, referentes às coletas do firewall e dos pontos de acesso, foram carregados e armazenados com sucesso pela etapa inicial do pipeline.

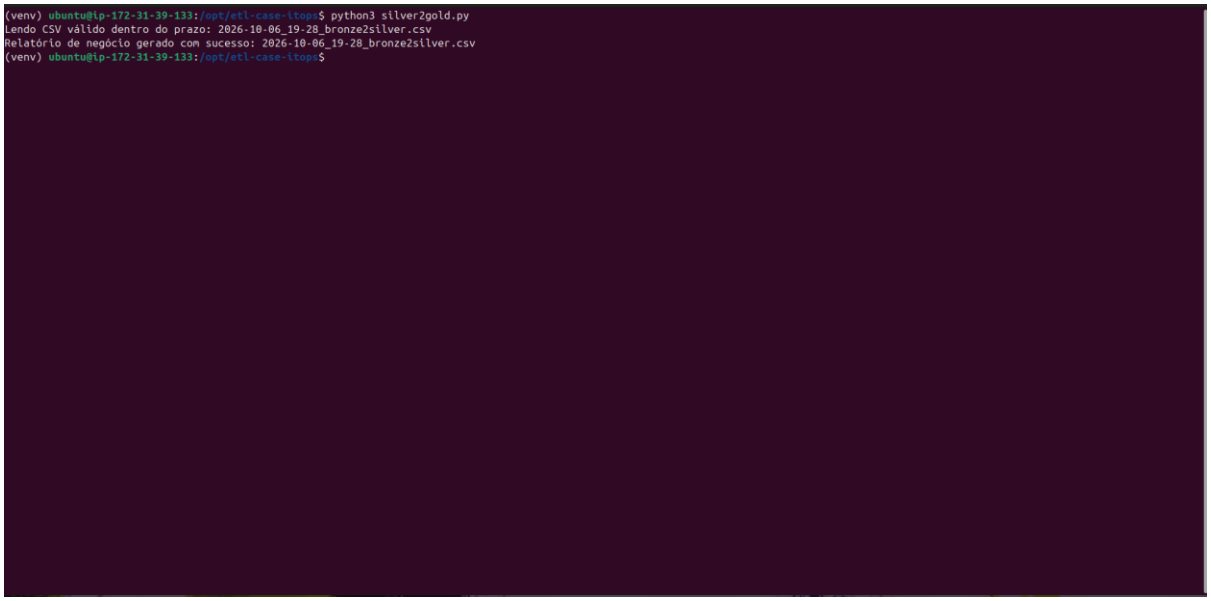
10. Criando CSV a partir dos arquivos python na instância



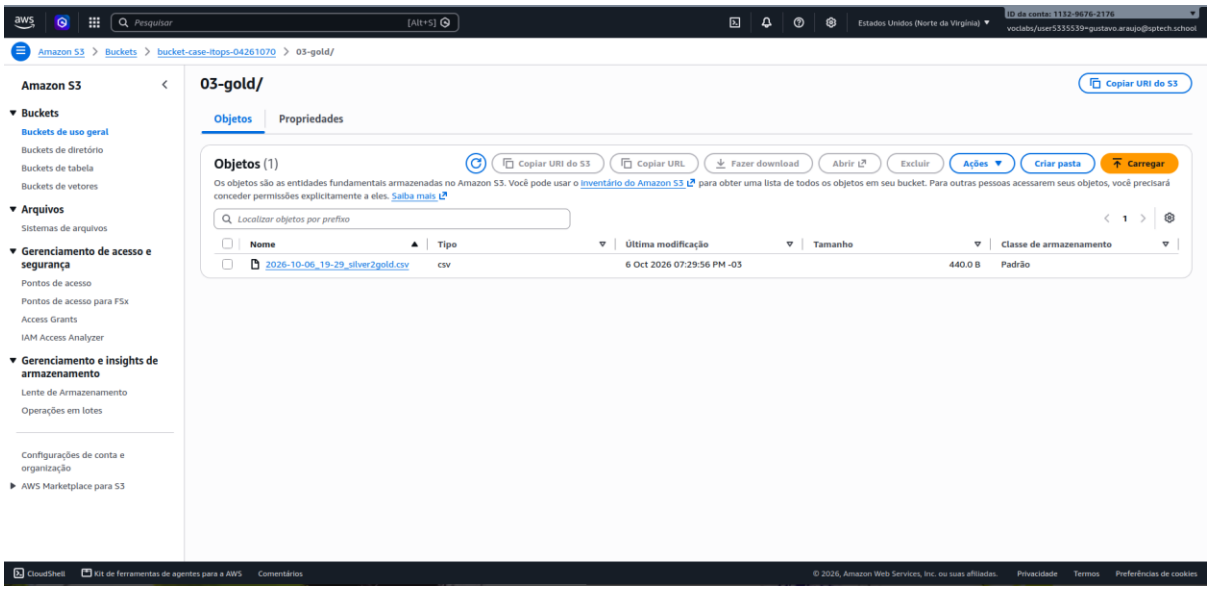
Retorno ao terminal da instância EC2, no diretório /opt/etl-case-itops, para acionar a primeira camada de transformação de dados. Executo o script bronze2silver.py, que consolida os arquivos JSON processados e retorna imediatamente a mensagem de sucesso, indicando a criação do relatório consolidado e seu envio direto para a camada silver.



Para verificar o resultado da transformação, navego até a pasta 02-silver/ dentro do bucket no console do Amazon S3. A tela confirma a presença do arquivo recém-gerado com a extensão .csv, atestando que os dados brutos foram adequadamente formatados, limpos e salvos para a próxima etapa analítica.

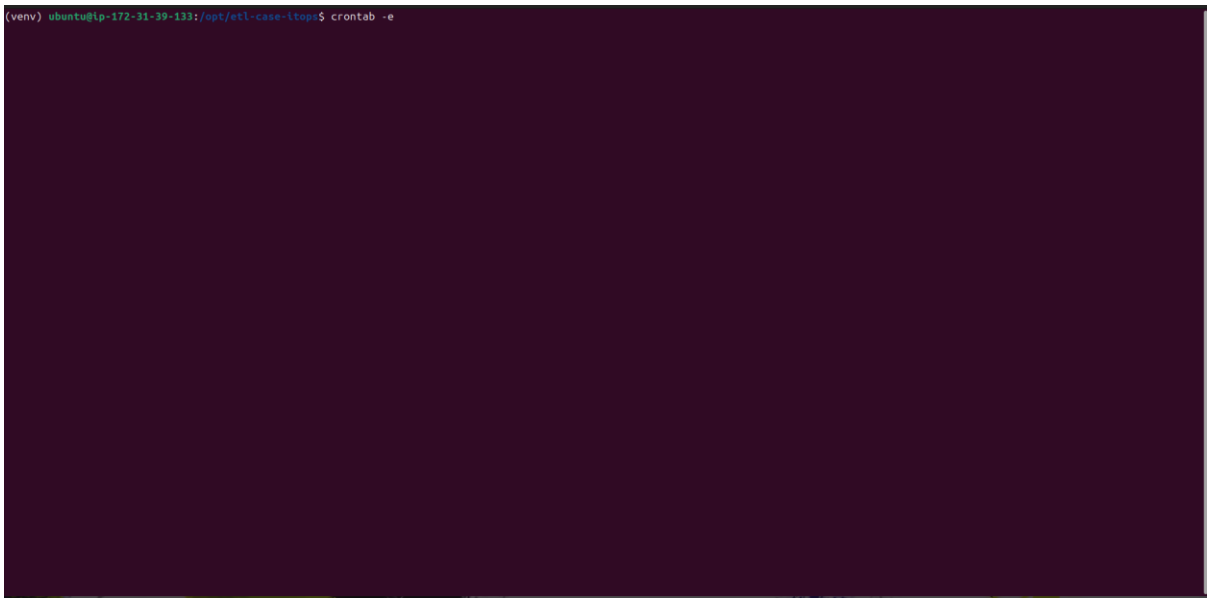


De volta ao terminal SSH da instância, aciono a etapa final do ETL executando o comando python3 silver2gold.py. O sistema exibe o log informando a leitura do CSV recém-criado dentro do prazo válido e sinaliza que a consolidação foi bem-sucedida, gerando o relatório final de negócios a partir da camada anterior.

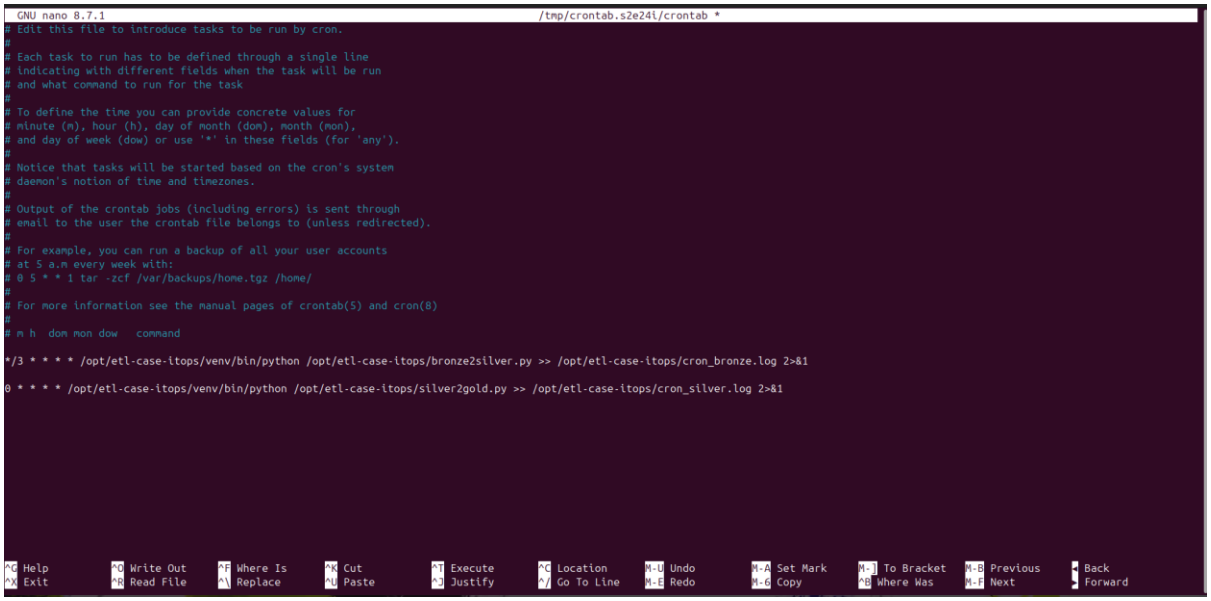


No console da AWS S3, abro o diretório 03-gold/ para validar a entrega final do fluxo de dados. Confirmo o armazenamento do arquivo 2026-10-06_19-29_silver2gold.csv, garantindo que os dados agora estão totalmente refinados, unificados e prontos para o consumo por ferramentas analíticas ou painéis executivos.

11. Configurando o contab



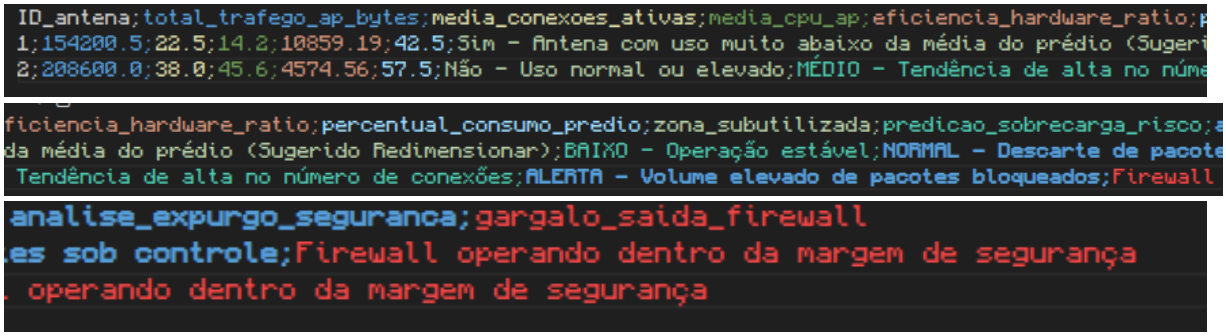
No terminal da instância EC2, apresento a execução do comando crontab -e dentro do diretório /opt/etl-case-itops. Este comando é o gatilho responsável por abrir e editar a tabela de tarefas cron do usuário, sendo o passo inicial e indispensável que permitiu estabelecer o agendamento contínuo dos scripts detalhado na tela de edição do nano.



Na interface do editor GNU nano, configuro a automação do pipeline através do arquivo de tarefas do sistema Linux. Defino duas rotinas temporizadas que apontam diretamente para o interpretador Python do ambiente virtual: a primeira agenda o script bronze2silver.py para rodar a cada 3 minutos, enquanto a segunda agenda o silver2gold.py para rodar pontualmente a cada hora, com ambas direcionando os logs de execução e erros para arquivos locais.

12. CSV Final

O CSV gerado na camada Gold resolve as necessidades de negócio estruturando as métricas brutas em colunas de decisão direta. Um exemplo de como ficou o CSV Final:



A coluna zona_subutilizada aponta claramente que a Antena 1 está subutilizada (menor tráfego geral), gerando o alerta de que a área tem uso muito abaixo da média, fundamentando a decisão de remanejamento físico do equipamento para economizar energia.

Através da coluna predicao_sobrecarga_risco, o relatório indica que a Antena 2 apresenta um risco "MÉDIO", sinalizando uma tendência de alta no número de conexões. Isso entrega exatamente o aviso preventivo para o time de ITOps agir antes que a rede trave.

A coluna eficiencia_hardware_ratio compara o tráfego com o uso de CPU. A Antena 1 é altamente eficiente (score de ~10859), enquanto a Antena 2 tem baixa eficiência (score de ~4574), revelando que ela consome muita CPU (45.6%) em proporção ao que trafega, indicando um possível equipamento obsoleto ou falha de firmware.

A coluna analise_expurgo_seguranca monitora os pacotes descartados. O CSV dispara um alerta claro para a Antena 2 ("Volume elevado de pacotes bloqueados"), alertando a equipe de cibersegurança sobre uma possível anomalia, como uma tentativa de ataque DDoS neste setor.

O direcionamento de investimentos é respondido pela coluna percentual_consumo_predio. Os dados cruzados mostram que o setor coberto pela Antena 2 consome a maior fatia da internet da empresa (57,5%), indicando que é lá onde o maior volume de usuários ou sistemas pesados estão operando.

Através da coluna gargalo_saida_firewall, o relatório diagnostica a saúde do equipamento central. Neste momento, a métrica informa que o "Firewall está operando dentro da margem de segurança", provando para a diretoria que o hardware ainda suporta a carga de regras atuais sem causar lentidão geral na rede.